

CUSTOMIZE BEHAVIOR

Plugins in the SDK

Load custom plugins to extend Claude Code with skills, agents, hooks, and MCP servers through the Agent SDK

Plugins allow you to extend Claude Code with custom functionality that can be shared across projects. Through the Agent SDK, you can programmatically load plugins from local directories to add skills, agents, hooks, and MCP servers to your agent sessions.

What are plugins?

Plugins are packages of Claude Code extensions that can include:

- **Skills:** Model-invoked capabilities that Claude uses autonomously (can also be invoked with `/skill-name`)
- **Agents:** Specialized subagents for specific tasks
- **Hooks:** Event handlers that respond to tool use and other events
- **MCP servers:** External tool integrations via Model Context Protocol

ⓘ The `commands/` directory is a legacy format. Use `skills/` for new plugins. Claude Code continues to support both formats for backward compatibility.

For complete information on plugin structure and how to create plugins, see [Plugins](#).

Loading plugins

Load plugins by providing their local file system paths in your options configuration. The `type` field must be `"local"` , the only value the SDK accepts. To use a plugin distributed through a [marketplace](#) or remote repository, download it first and provide the local directory path. The SDK supports loading multiple plugins from different locations.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Hello",
  options: {
    plugins: [
      { type: "local", path: "./my-plugin" },
      { type: "local", path: "/absolute/path/to/another-plugin" }
    ]
  }
})) {
  // Plugin commands, agents, and other features are now available
}
```

Path specifications

Plugin paths can be:

- **Relative paths:** Resolved relative to your current working directory (for example, `./plugins/my-plugin`)
- **Absolute paths:** Full file system paths (for example, `/home/user/plugins/my-plugin`)

❗ The path should point to the plugin's root directory: the parent of `skills/` , `agents/` , `hooks/` , `commands/` (legacy), or `.claude-plugin/` , not a subdirectory.

Verifying plugin installation

When plugins load successfully, they appear in the system initialization message. You can verify that your plugins are available:

```

import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Hello",
  options: {
    plugins: [{ type: "local", path: "./my-plugin" }]
  }
})) {
  if (message.type === "system" && message.subtype === "init") {
    // Check loaded plugins
    console.log("Plugins:", message.plugins);
    // Example: [{ name: "my-plugin", path: "./my-plugin" }]

    // Plugin skills appear with the plugin name as a prefix
    console.log("Skills:", message.skills);
    // Example: ["my-plugin:greet"]

    // Plugin commands use the same prefix, and skills appear here too
    console.log("Commands:", message.slash_commands);
    // Example: ["compact", "context", "my-plugin:custom-command", "my-
plugin:greet"]
  }
}

```

Using plugin skills

Skills from plugins are automatically namespaced with the plugin name to avoid conflicts. To invoke one directly, send `/plugin-name:skill-name` as the prompt.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Load a plugin with a custom /greet skill
for await (const message of query({
  prompt: "/my-plugin:greet", // Use plugin skill with namespace
  options: {
    plugins: [{ type: "local", path: "./my-plugin" }]
  }
})) {
  // Claude executes the custom greeting skill from the plugin
  if (message.type === "assistant") {
    console.log(message.message.content);
  }
}
```

❗ If you installed a plugin via the CLI (for example, `/plugin install my-plugin@marketplace`), you can still use it in the SDK by providing its installation path. Check `~/.claude/plugins/` for CLI-installed plugins.

Complete example

Here's a full example demonstrating plugin loading and usage:

```

import { query } from "@anthropic-ai/claude-agent-sdk";
import * as path from "path";

async function runWithPlugin() {
  const pluginPath = path.join(__dirname, "plugins", "my-plugin");

  console.log("Loading plugin from:", pluginPath);

  for await (const message of query({
    prompt: "What custom commands do you have available?",
    options: {
      plugins: [{ type: "local", path: pluginPath }],
      maxTurns: 3
    }
  })) {
    if (message.type === "system" && message.subtype === "init") {
      console.log("Loaded plugins:", message.plugins);
      console.log("Available skills:", message.skills);
      console.log("Available commands:", message.slash_commands);
    }

    if (message.type === "assistant") {
      console.log("Assistant:", message.message.content);
    }
  }
}

runWithPlugin().catch(console.error);

```

Plugin structure reference

A plugin directory typically contains a `.claude-plugin/plugin.json` manifest file. The manifest is optional. When omitted, Claude Code auto-discovers components from the directory layout. The directory can include:

```

my-plugin/
├── .claude-plugin/
│   └── plugin.json          # Plugin manifest (optional,
components auto-discovered without it)
├── skills/                  # Agent Skills (invoked
autonomously or via /skill-name)
│   └── my-skill/
│       └── SKILL.md
├── commands/                # Legacy: use skills/ instead
│   └── custom-cmd.md
├── agents/                  # Custom agents
│   └── specialist.md
├── hooks/                   # Event handlers
│   └── hooks.json
└── .mcp.json                # MCP server definitions

```

For detailed information on creating plugins, see:

- [Plugins](#) - Complete plugin development guide
- [Plugins reference](#) - Technical specifications and schemas

Common use cases

Development and testing

Load plugins during development without installing them globally:

```
plugins: [{ type: "local", path: "./dev-plugins/my-plugin" }];
```

Project-specific extensions

Include plugins in your project repository for team-wide consistency:

```
plugins: [{ type: "local", path: "./project-plugins/team-
workflows" }];
```

Multiple plugin sources

Combine plugins from different locations:

```
plugins: [  
  { type: "local", path: "./local-plugin" },  
  { type: "local", path: "~/.claude/custom-plugins/shared-plugin"  
}  
];
```

Troubleshooting

Plugin not loading

If your plugin doesn't appear in the init message:

1. **Check the path:** ensure the path points to the plugin root directory, the parent of `skills/` , `agents/` , `hooks/` , `commands/` (legacy), or `.claude-plugin/`
2. **Validate plugin.json:** if your plugin includes a manifest, ensure it has valid JSON syntax
3. **Check file permissions:** ensure the plugin directory is readable

Skills not appearing

If plugin skills don't work:

1. **Use the namespace:** invoke plugin skills as `/plugin-name:skill-name`
2. **Check init message:** verify the skill appears in the `skills` list with the correct namespace
3. **Validate skill files:** ensure each skill has a `SKILL.md` file in its own subdirectory under `skills/` , for example `skills/my-skill/SKILL.md`

Path resolution issues

If relative paths don't work:

1. **Check working directory:** Relative paths are resolved from your current working directory
2. **Use absolute paths:** For reliability, consider using absolute paths
3. **Normalize paths:** Use path utilities to construct paths correctly

See also

- **Plugins** - Complete plugin development guide
- **Plugins reference** - Technical specifications
- **Commands** - Using commands in the SDK
- **Subagents** - Working with specialized agents
- **Skills** - Using Agent Skills